

Real-World Bug Hunting

Learning Sources

[Real-World Bug Hunting - A Field Guide to Web Hacking](#)

Notes

-Notes originally created in Sept 2025

01. Bug Bounty Basics

Summary

-This chapter focuses on some basic web concepts, such as vulnerabilities, bug bounties, servers, IP addresses, HTTP, etc.

Vulnerabilities & Bug Bounties

-Vulnerability – A weakness in an app that allows a malicious actor to perform an non-permitted app action or gain access to data they should not have access to.

-A vulnerability can result in an attacker performing intended and unintended actions. Unintended action example: changing ID of record to access some info.

-Even actions which do not required “hacking” that let a user do/access something they should not is a vulnerability

-Bug Bounty – A reward (cash, crypto, airline miles, etc.) a website owner gives to someone who ethically discovers a bug and reports it to owner.

-Bug Program – An app owner’s rules and frameworks for people who want to test for vulnerabilities in an “ethical” way

-Vulnerability Disclosure Program – A program to report bugs that offers no bounty

Client / Server

-Internet – Network of computers that send messages to each other

-Packets – Data and metadata sent across the net. Servers can limit the types of packets they accept. Used to split data into small parts and can hence request only part of full data again if packet is lost.

-Client – initiates requests

-Server – receives request and responds

-HTTP – An RFC which defines how internet browsers communicate w/ a remote server using Internet Protocol

Visiting a Website

1. Visit site in browser (ex. <https://google.com>) and send request for site to ISP
2. ISP resolves domain name (ex. google.com) to an IP address (ex. 142.251.45.206) via DNS (Domain Name System) which store *domain:ip* refs
3. ISP establishes a TCP (Transmission Control Protocol) w/ server on port 80 (common HTTP port) or port 443 (common HTTPS port) to establish a 2-way comm between client and server

4. HTTP request passed to server by ISP
5. Server processes request and responds
6. ISP sends to client
7. Browser renders response

-IPv4 – Four nums connected by periods in ranges 0-255 (ex. *167.160.50.01*)

-IPv6 – Eight groups of hexadecimal digits separated by colons (ex. *2001:4860:4860::8888*). Made as num of available IPv4 addresses are limited.

-Ex HTTP request:

GET / HTTP/1.1

Host: *www.google.com*

Connection: keep-alive

Accept: application/html, */*

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)

-Path – A specific endpoint on a site (ex. *mysite.com/my_path*)

-Only skimmed the rest of this chapter and skipped notes as I already know all this. For more detailed info on HTTP on basic client/server, see existing notes files: *HTTP Notes*, *FastAPI Notes*, and *TryHackMe Notes*, *Web Dev Prog Lang Overview*, and *HTML Notes*.

02. Open Redirect

Intro

-Open Redirect – A vulnerability which occurs when a target visits a site and that site re-directs to a diff URL, potentially on a diff domain, as a way to lure targets to a malicious site.

-Often used with phishing, to make client think they are submitting info to actual site, when is actually malicious copy

-Can be low impact and not deserving of a bounty. Not part of OWASP (Open Web Application Security Project) Top 10.

Param Redirects

-Occur when a dev trusts attack-controlled input to redirect to another site, usually via URL param, HTML refresh tags, or DOM window location prop.

-Param based ex.: *https://www.google.com/?redirect_to=https://www.gmail.com*

-When re-direct occurs original client HTTP message is sent to site that is being re-directed to

-To look for open re-directs, generally monitoring proxy history for GET requests sent to ur test site that include a param specifying URL redirects.

Example Param OR Attack

-What if Google does not check re-directs before re-directing and attacker substitutes actual Google hyperlink on some site to *https://www.google.com/?redirect_to=https://www.attacker.com*? Now sensitive info intended for Google can go to attacker.

-Such vulnerabilities often include params such as *url=*, *redirect=*, *next=*, etc., so watch for this in URL

HTML <meta> Redirects

-<meta> - HTML tag which can define redirects.

-Ex. <meta http-equiv="refresh" content="0; url=https://www.attacker-site.com/">

-content – attribute that tells how long to wait before making request to URL in seconds

-Both *url* and *content* can be used as parts of attack

JavaScript Redirects

-Can also re-direct by modifying the DOM's *location* prop via JS:

```
window.location = https://www.google.com/           # all three re-direct  
window.location.href = https://www.google.com  
window.location.replace(https://www.google.com)
```

-JS redirects require an attacker to be able to execute JS, such as via XSS

Example: Shopify Theme Install OR

-Bug Report: <https://hackerone.com/reports/101962>

-In this \$500 low difficulty bug bounty found for Shopify in 2015, the following open re-direct was found:

https://apps.shopify.com/services/google/themes/preview/supply--blue?domain_name=<anydomain>

-While the intended effect was giving a way for Shopify store admins to see a preview of a new theme for their store, the result allowed attackers to submit malicious *anydomain*, instead of just the users store domain, which then would re-direct the user to a malicious site.

Example Shopify Login OR

-Bug Report: <https://hackerone.com/reports/103772>

-In another \$500 low difficulty bug bounty found for Shopify in 2015, the following open re-direct was found:

http://mystore.myshopify.com/account/login?checkout_url=.attacker.com

-The intended effect was to re-direct a user to a specific Shopify store URL (ex. checkout) after user logged in. But it also allowed attackers to re-direct to a malicious URL after login, as well. The attacker could then steal login info, and so forth.

-You can add special chars (ex. period in above) to part of a URL to redirect to another domain

Example HackerOne Interstitial OR

-Bug Report: <https://hackerone.com/reports/111968>

-In this \$600 low difficulty bug bounty found for HackerOne in 2016, the following open re-direct was found

-Interstitial Web Page – a page which displays before the expected content loads to prevent open re-directs. Can be used to let user know they are leaving the current domain, also.

-Can thus have the interstitial web page re-direct to a malicious page, such as fake login page.

-In HackerOne example, HO used ticketing system Zendesk and when took HO to path */zendesk_session*, the browser re-directed to Zendesk, with an interstitial web page in between. But then users could create custom

Zendesk accounts and pass them to the `/redirect_to_account?state=param`. The custom Zendesk acct could then re-direct to malicious website which would take user to untrusted site w/o warning.

-This vulnerability was confirmed via a hacker creating a Zendesk acct under subdomain *compayn.zendesk.com*, then adding the following to Zendesk theme: `<script>document.location.href = «http://evil.com»;</script>`. As such HackerOne would re-direct to Zendesk, which would then run his script to re-direct to the malicious site via:

`https://hackerone.com/zendesk_session?locale_id=1&return_to=https://support.hackerone.com/ping/redirect_to_account?state=compayn:/`

-Because the *return_to* re-direct contained the *hackerone* domain, the interstitial web page never loaded and users never knew they were being re-directed to malicious site.

-Each service a site uses is thus a new attack vector. Here, though Zendesk re-directs were allowed and sent through an interstitial site first, due to Zendesk vulnerabilities, the attacker was able to exploit HackerOne also. A vulnerability sometimes must thus be dug for, through multiple layers, to find.

Summary

-Open redirects let attacks redirect clients to malicious sites without clients knowing. They can occur through obvious redirect params or more hidden ways.

-The attack relies on abuse of trust where the client does not realize the site they have been redirected to is malicious

-Integrating services into your service can expose more vulnerabilities